

NAME: GALLA MOHAN SAI

MODULE NO : 1

Exercise 1: Implementing the Singleton Pattern

DatabaseConnection.java

```
public class DatabaseConnection {

    private static DatabaseConnection instance;

    private DatabaseConnection() {
        System.out.println("Database Connection Created!");
    }

    public static DatabaseConnection getInstance() {
        if (instance == null) {
            instance = new DatabaseConnection();
        }
        return instance;
    }

    public void connect() {
        System.out.println("Connected to Database");
    }
}
```

DatabaseTest.java

```
public class DatabaseTest {
```

```

public static void main(String[] args) {

    DatabaseConnection db1 = DatabaseConnection.getInstance();

    DatabaseConnection db2 = DatabaseConnection.getInstance();

    db1.connect();

    db2.connect();

    if (db1 == db2) {

        System.out.println("Only one DatabaseConnection instance exists.");

    } else {

        System.out.println("Multiple instances exist.");

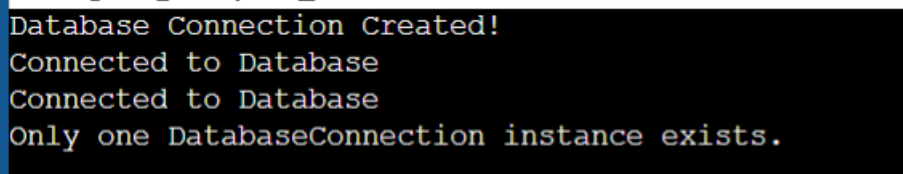
    }

}

}

```

OUTPUT:



```

Database Connection Created!
Connected to Database
Connected to Database
Only one DatabaseConnection instance exists.

```

Exercise 2: Implementing the Factory Method Pattern

Notification.java

```

public interface Notification {
    void send();
}

```

EmailNotification.java

```

public class EmailNotification implements Notification {

    public void send() {
        System.out.println("Sending Email Notification");
    }
}

```

```
}  
}
```

SMSNotification.java

```
public class SMSNotification implements Notification {  
  
    public void send() {  
        System.out.println("Sending SMS Notification");  
    }  
}
```

PushNotification.java

```
public class PushNotification implements Notification {  
  
    public void send() {  
        System.out.println("Sending Push Notification");  
    }  
}
```

NotificationFactory.java

```
public abstract class NotificationFactory {  
  
    public abstract Notification createNotification();  
}
```

EmailFactory.java

```
public class EmailFactory extends NotificationFactory {  
  
    public Notification createNotification() {  
        return new EmailNotification();  
    }  
}
```

SMSFactory.java

```
public class SMSFactory extends NotificationFactory {  
  
    public Notification createNotification() {  
        return new SMSNotification();  
    }  
}
```

PushFactory.java

```
public class PushFactory extends NotificationFactory {  
  
    public Notification createNotification() {  
        return new PushNotification();  
    }  
}
```

FactoryMethodTest.java

```
public class FactoryMethodTest {  
  
    public static void main(String[] args) {  
  
        NotificationFactory emailFactory =  
            new EmailFactory();  
  
        Notification email =  
            emailFactory.createNotification();  
  
        email.send();  
  
        NotificationFactory smsFactory =  
            new SMSFactory();  
  
        Notification sms =  
            smsFactory.createNotification();  
  
        sms.send();  
  
        NotificationFactory pushFactory =  
            new PushFactory();  
  
        Notification push =  
            pushFactory.createNotification();  
  
        push.send();  
    }  
}
```

OUTPUT:

